

Thread ինչ է — Խորը ուղեցույց

C++ Concurrency presentation

Այս ուղեցույցը մանրամասն բացատրում է thread-ի՝ ինչ է, process vs thread, ինչպես steghcel (std::thread), join/detach, orinaknerov ev interview harcerov.

1. Ի՞նչ է Thread-ը

Thread-ի ("threadic") program-ի **amenapoqr kataroghakan miavor** է. Mek process-ի mej karox en linel **mi qani thread** vor ashxatum en zugaher (parallel/concurrent).

Process = ashxatox program (uni ir memory space)

Thread = process-ի mej kataroghakan hosq (kisum e process-ի memory-n)

```
#include <thread>
#include <iostream>

void task() { std::cout << "Hello from thread\n"; }

int main() {
    std::thread t(task);    // nor thread steghcum
    t.join();               // spasum minchev t avartvi
}
```

2. Process vs Thread

PROCESS:

- Uni AĀANCIN memory space
- Canr (steghcumy tañk)
- Meko myusin chi azdum (isolated)
- Communication: IPC (pipes, sockets)

THREAD:

- KISUM e process-i memory-n (shared)
- Teṗ (steghcumy ejan)
- Mek thread-i sxaly karoḡ e taputakel amboxjy
- Communication: shared memory (unix)

	Process	Thread
Memory	aĀancin	kisvac (shared)
Steghcum	tañk	ejan
Isolation	ayo	voch
Communication	IPC	shared memory
Crash azdecutyun	isolated	amboxjy

3. Thread steghcum (std::thread)

Function-ov

```
void work(int id) { cout << "Thread " << id << "\n"; }  
std::thread t(work, 1); // argument-ner  
t.join();
```

Lambda-ov

```
std::thread t([]() { cout << "lambda thread\n"; });  
t.join();
```

Mi qani thread

```
std::vector<std::thread> threads;
for (int i = 0; i < 5; i++)
    threads.emplace_back([i]() { cout << "Thread " << i << "\n"; });
for (auto& t : threads) t.join();
```

4. join() vs detach()

```
std::thread t(task);
t.join();      // spasum e minchev thread-y avartvi (blocking)
```

```
std::thread t(task);
t.detach();    // thread-y ashxatum e ankax (background), cheng spasum
```

```
join()    -> spasum ekavartin (sovorabar sa)
detach()  -> ankax (daemon-i pes), cheng karoxtanum join anel heto
```

Karevor. thread object-y ochnceluc araj petq e kam `join()` kam `detach()` kanchel, hakaarak depqum -> `std::terminate` (crash).

5. Thread-i vtangnery (nerhatuk)

Qani or thread-nery kisum en memory-n, arajanum en problem-ner.

```
Race Condition  -> mi qani thread miasin poxum en nuyn data
Deadlock        -> thread-ner spasum en irar -> kaxvac
Data Race       -> synchronization-ic aranc shared data mutq -> UB
```

Sranq lucvum en `mutex`, `lock_guard`, `atomic` -ov (tes arancin presentation-ner).

6. Inchu Thread-ner

- + Parallelism (mi qani CPU core -> aveli arag)
- + Responsiveness (UI chi sařum erkar operacia-i zham)
- + Resource sharing (nuyn memory -> arag communication)
- Complexity (race condition, deadlock)
- Debugging dzhvar (non-deterministic)

7. Interview harcer & patasxanner

Thread inch e?

Program-i amenapoqr kataroghakan miavor; mek process-i mej mi qani thread, kisum en memory-n.

Process vs Thread?

Process ařancin memory, canr, isolated. Thread shared memory, teř, mek process-i mej.

join vs detach?

join spasum e thread-i avartin; detach thread-y darnum ankax (background).

Inchu petq e join/detach?

thread object ochnceluc ařaj; hakařak depqum std::terminate.

Thread-i himnakan problem-nery?

Race condition, deadlock, data race (shared memory-i patchařov).

Thread inchi hamar?

Parallelism (multi-core), responsiveness, resource sharing.

Cheat Sheet

THREAD = program-i amenapoqr kataroghakan miavor, kisum process-i memory

PROCESS vs THREAD:

process -> arāncin memory, canr, isolated, IPC

thread -> shared memory, tēp, mek process, shared-memory communication

std::thread t(func, args) | t.join() (spasum) | t.detach() (ankax)

ochncumic arāj -> join/detach (aylapes std::terminate)

PROBLEMS: race condition | deadlock | data race -> mutex/atomic

USE: parallelism | responsiveness | resource sharing